Abstract Interpretation: An Annotated Bibliography

Andre Quimper Osores (aquimper@umich.edu)
University of Michigan

April 13, 2026

1 Introduction and Motivation

The verification of software systems is fundamentally constrained by **Rice's Theorem**, which states that any non-trivial semantic property of a program is undecidable. As software grows and becomes increasingly integrated into safety-critical infrastructure, from flight control systems to medical devices, our inability to predict program behavior creates a massive risk. **Abstract Interpretation** addresses this challenge by shifting the goal from "exact" verification to **sound approximation**. We model a program in an abstract domain that simplifies the state space while preserving properties that we care about. This framework provides a mathematical guarantee: If the analyzer proves a property in the abstract domain, then it must also hold in the concrete domain, effectively trading completeness for sound, decidable results.

2 Evolution of the Field

The foundation of the field was established by Cousot and Cousot (1977, 1979), who introduced a unified lattice model for static analysis. They formalized the relationship between concrete program semantics and abstract domains using **Galois Connections**. A Galois Connection is a pair of adjoint functions: abstraction (α), which maps a set of concrete program states to the smallest element in the abstract domain that contains them, and concretization (γ), which maps an abstract element back to the set of concrete states it represents.

For a pair to be a "sensible" representation, they must satisfy the property:

$$\alpha(c) \leq a \iff c \subseteq \gamma(a)$$

This ensures that the abstraction is always sound—it never omits a possible concrete behavior—and that $\gamma \circ \alpha$ is extensive (meaning the round-trip from concrete to abstract and back results in a set at least as large as the original). For example, in an *Interval Analysis*, a set of concrete integers $\{1, 2, 5\}$ would be abstracted via α to the interval $[1, 5]$. While we lose the information that 3 and 4 were not in the original set, the interval allows us to efficiently compute bounds for arithmetic operations using simple box algebra rather than tracking individual points in an infinite state space.

Formally defining the abstraction and concretization function allowed researchers to systematically derive the "best" abstract transformers. Crucially, they introduced widening and narrowing operators to ensure that fixpoint computations would terminate even in infinite-height lattices, providing a rigorous mathematical backbone for all subsequent static analysis tools.

As the theoretical framework matured, the focus shifted toward developing specific abstract domains capable of capturing numerical relationships. Researchers sought to prove properties like the

absence of buffer overflows, division-by-zero errors, and out-of-bounds array accesses. To do this, an analyzer must track the possible values variables can take using numerical abstract domains. These are categorized into non-relational domains, such as Intervals, which track each variable independently ($x \in [0, 10]$), and relational domains, which capture mathematical dependencies between variables ($x \leq y$). Non-relational domains are computationally efficient but imprecise; for example, they cannot prove that $x - y = 0$ even if x and y are equal. In contrast, relational domains allow the analyzer to verify complex safety properties, such as ensuring an index always stays within the bounds of a dynamically allocated buffer or proving that linear relationships between sensor inputs and actuator outputs remain stable.

The **Polyhedra domain** (1978) was a landmark achievement, allowing the discovery of linear inequalities between multiple variables (e.g., $x + y \leq z$). Such relational information is vital for verifying loop invariants where one variable's bound depends on another. While highly precise, the exponential computational cost of polyhedra created a scalability barrier. This gap was later addressed by Antoine Miné (2006) with the **Octagon** abstract domain. By restricting constraints to the form $\pm x \pm y \leq c$, Miné utilized Difference-Bound Matrices and shortest-path algorithms to achieve $O(n^3)$ time complexity, providing a "weakly relational" middle ground that balanced the efficiency of simple intervals with the precision of polyhedra.

In the 1990s, the community revisited the core mechanics of convergence. Cousot and Cousot (1992) defended the necessity of widening and narrowing, proving that no single finite lattice could capture all necessary invariants for every program. This era reinforced the idea that infinite abstract domains, combined with clever extrapolation (widening), were more powerful than restricted finite models. By 1996, the field had reached sufficient maturity to begin its "industrialization" phase, shifting from academic proofs to the creation of commercial-grade tools like AbsInt and Polyspace to handle the increasing complexity of modern C code.

The greatest achievement of this evolution was the development of the **Astrée Static Analyzer** (2003, 2005). Designed specifically for the safety-critical primary flight control software of the Airbus A340 and A380, Astrée demonstrated that abstract interpretation could scale to hundreds of thousands of lines of code. By employing a "design by refinement" strategy—integrating specialized domains like *Ellipsoids* to bound the rounding errors in digital filters and using widening with thresholds to maintain precision in loops—the team achieved zero false alarms on massive ($\sim$400 Thousand LoC), industrial-scale codebases.

3 Summary of Open Research Questions

Despite its industrial success, several questions in abstract interpretation remain unanswered. Current research is focused on **concurrency and asynchrony**, as most models (including the original Astrée) only work for synchronous, single-threaded execution. Theorem-prover based methods can also be applied statically and produce similar information as Abstract Interpretation; recent research has begun to explore the connection between these logic-based methods and the lattice-based framework of Abstract Interpretation. Finally, automated generation of sophisticated widening operators by integrating Machine Learning and SMT/SAT Solvers is being explored to reduce the manual effort currently required to tune analyzers for specific application domains.

References

[1] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th POPL*, 1977.

Problem: The authors address the lack of a unified framework for program analysis, specifically the challenge of relating actual program computations in a universe of objects to "abstract objects" that provide meaningful information about those computations.
Solution: The paper formalizes abstract interpretation as a 6-tuple $I = \langle A\text{-}Cont, \sqcup, \leq, \top, \bot, Int \rangle$ within a complete lattice. It establishes that the resulting context vector is the solution to the fixpoint equation $Cv = Int(Cv)$. The framework introduces Galois connections—defined by abstraction (α) and concretization (γ) functions—to ensure consistency between concrete and abstract interpretations. To handle termination in infinite-height lattices, the paper proposes the use of Kleene's Sequence along with widening operators to accelerate convergence and narrowing operators to refine precision.
Evaluation: The methodology is validated through rigorous formal mathematical proofs of the underlying lattice models and fixpoint construction theorems.
Limitations: The authors identify a need for measures to control result accuracy and acknowledge that approximation methods do not always guarantee the discovery of the specific fixpoint of interest, requiring further fixpoint improvement methods.

[2] P. Cousot and N. Halbwachs, "Automatic discovery of linear restraints among variables of a program," in *Proc. 5th POPL*, 1978.

Problem: The need for static determination of linear equality and inequality relations among variables without user-provided assertions. Previous techniques like constant propagation were limited to simple relations (e.g., $x \in [a, b]$), failing to capture interdependent constraints between multiple variables.
Solution: The introduction of the *Polyhedra* abstract domain. Programs are modeled as flowcharts where each arc is associated with a closed convex polyhedron in $\mathbb{R}^n$. The paper defines transformers for assignments, tests, and junctions using dual representations (linear inequalities and "frames" of vertices/rays). It also introduces the critical widening and narrowing operators to ensure fixpoint convergence in infinite-height lattices.
Evaluation: Provided through formal mathematical proofs of the dual-representation conversion algorithms (e.g., Lanery's method). The effectiveness of the approach is demonstrated via "convincing examples" such as the HEAPSORT algorithm, showing the discovery of non-trivial inductive invariants.
Limitations: The conversion between systems of restraints and frames is computationally expensive, often resulting in exponential complexity relative to the number of constraints. Additionally, the domain is strictly limited to linear relationships and cannot capture non-linear restraints (e.g., $x = y^2$).

[3] P. Cousot and R. Cousot, "Systematic design of program analysis frameworks," in *Proc. 6th POPL*, 1979.

Problem: The challenge of defining an abstract domain $\overline{A}$ such that every property has a unique "best" interpretation relative to concrete semantics A. It addresses

how to derive an abstract program interpretation that is guaranteed to be the most precise possible approximation while ensuring fixpoint computations remain sound and consistent with the ideal "Meet Over all Paths" (MOP) result.
Solution: The authors prove that the abstract domain must be a Moore family, allowing for the definition of an upper closure operator $\rho(P) = \bigwedge\{Q \in \overline{A} \mid P \Rightarrow Q\}$. They formalize Galois connections via a pair of adjoined functions (abstraction α and concretization γ) such that $\alpha(p) \sqsubseteq \bar{p} \iff p \subseteq \gamma(\bar{p})$. This allows the "best" abstract transformer t to be systematically derived as $t = \alpha \circ \tau \circ \gamma$.
Evaluation: Soundness is established by proving the least fixpoint of the abstract equations is a sound approximation of the concrete semantics. The paper identifies the conditions under which the fixpoint solution is equal to the MOP result and demonstrates how to construct complex analyzers from simple ones using reduced products of lattices.
Limitations: While the "best" transformer always exists theoretically, it may not be efficiently computable, often requiring a coarser but more efficient alternative in practice. The authors suggest that future work in relational abstractions (like polyhedra) will require specialized techniques such as widening and narrowing to maintain relationships between variables.

[4] P. Cousot and R. Cousot, "Comparing galois connections and widening/narrowing," in *Proc. PLILP '92*, 1992.

Problem: A general preference for the Galois Connection approach over widening/narrowing due to the latter being less understood, despite widening/narrowing being more general. The core issue is that finite lattices often fail to capture necessary invariants, while fixed point computations on infinite lattices may fail to converge without a formal mechanism to accelerate termination.
Solution: The authors demonstrate that widening and narrowing on infinite abstract domains provide a more powerful framework than Galois Connections restricted to finite lattices. Widening is employed during upward iterations to force convergence by extrapolating and intentionally losing information to ensure termination. Narrowing is then applied to the resulting over-approximation, performing a downward iteration to recover precision while maintaining safety.
Evaluation: The paper establishes four landmark theorems: (1) for any specific program, there exists a finite lattice that yields results equivalent to widening/narrowing; (2) no single finite lattice can achieve this for all programs; (3) infinitely many abstract values are necessary to analyze all programs; and (4) the required abstract values for a specific program cannot be inferred by simple inspection of the source text.
Limitations: The selection of an effective widening operator remains largely heuristic; while generating a naive operator is trivial, it is often not useful for precision. Future work is suggested toward the automated generation of sophisticated widening and narrowing operators that can adapt to specific program structures.

[5] P. Cousot, "Abstract interpretation: Achievements and perspectives (survey)," in *Proc. SSGRR '00*, 1996.

Problem: A comprehensive survey addressing the increasing complexity of software and the need for automated error-detection techniques. It grounds the necessity of Ab-

stract Interpretation in Rice's Theorem, which dictates that because exact verification of non-trivial properties is undecidable, we must rely on formal, sound approximations.

Solution: The paper formalizes program execution as trace semantics and defines the verification process as checking if these traces satisfy a specification. It frames the abstraction of these traces as the solution to a fixpoint equation. To make this practical, it utilizes Galois Connections for sound mapping between concrete and abstract domains, using widening and narrowing operators to ensure these computations converge efficiently.

Evaluation: The maturity of the field is demonstrated by its industrialization. The author highlights the emergence of commercial static analysis tools from startups such as AbsInt Angewandte Informatik (Germany, 1998) and Polyspace Technologies (France, 1999), marking the shift from academic theory to industrial safety-critical applications.

Limitations: Significant hurdles remain for the "next generation" of analyzers, specifically the handling of complex control flow and data structures. There is a noted need for better compositional design (building large analyzers from smaller parts) and the ability to verify more intricate, high-level program properties.

[6] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, "A static analyzer for large safety-critical software," in *Proc. PLDI '03*, 2003.

Problem: The difficulty of formally proving the absence of runtime errors in large-scale, safety-critical software. Key challenges include achieving "zero false alarms" (high precision) while maintaining scalability and correctly modeling IEEE 754 floating-point arithmetic.

Solution: The development of Astrée, a static analyzer based on Abstract Interpretation. The approach uses a "design by refinement" strategy where domains are manually tuned to eliminate false alarm patterns. It introduces specialized numerical domains, such as Octagons and Ellipsoids, specifically to capture invariants in digital signal processing (DSP) filters.

Evaluation: Demonstrated through a case study on an industrial C program (Airbus A340 flight control software) consisting of 132,000 lines. The analyzer successfully proved the absence of all runtime errors (0 alarms) in 11 hours using 2GB of memory on a standard PC.

Limitations: The analyzer is specialized for a synchronous, non-recursive C subset without dynamic memory allocation. Additionally, significant manual effort is required for analyzer tuning, and standard backward slicing remains too imprecise for investigating complex invariants.

[7] P. Cousot *et al.*, "The astrée static analyzer," in *Proc. ESOP '05*, 2005.

Problem: The need for a modular and extensible framework for soundly verifying safety-critical embedded C code. The challenge lies in maintaining "zero false alarm" precision while scaling to significantly larger codebases (nearly half a million lines) and handling complex floating-point rounding errors.

Solution: A multi-abstraction architecture that integrates a collection of specialized numerical and symbolic domains (e.g., intervals, octagons, and digital filters). It utilizes widening/narrowing with thresholds to ensure termination and employs a

modular design that allows the analyzer to be tailored to specific application domains.
Evaluation: Evaluated on the Airbus A380 primary flight control software. The tool achieved a full correctness proof for a 400,000-line C program in under 12 hours (approx. 11h 48m) with zero false alarms. The authors also introduced a *Visualisator* tool to help users navigate the computed invariants.
Limitations: The analyzer remains restricted to synchronous programs without dynamic memory, recursion, or backward branching. Future work involves a total redesign of the memory abstraction to support asynchronous programs and integrating backward analysis for better error localization.

[8] A. Miné, "The octagon abstract domain," *Higher-Order and Symbolic Computation*, 2006.

Problem: The significant gap between the Interval domain (efficient but imprecise) and the Polyhedra domain (precise but with exponential cost). There was a need for a "weakly relational" domain that could capture relationships between variables more effectively than intervals without the prohibitive computational overhead of full polyhedra.
Solution: The introduction of the *Octagon* abstract domain, which captures constraints of the form $\pm x \pm y \leq c$. These are represented using *Difference-Bound Matrices* (DBMs) of size $2n \times 2n$ by encoding each variable v as two nodes (v^+ and v^-) in a potential graph. The paper defines algorithms for lattice operations using a modified Floyd-Warshall shortest-path algorithm to achieve a "Strong Closure" (normal form), ensuring the tightest possible bounds are discovered.
Evaluation: The paper provides formal theorems establishing the correctness and optimality of the operators, specifically focusing on coherence and the properties of the strong closure. The domain achieves a spatial complexity of $O(n^2)$ and a temporal complexity of $O(n^3)$ per abstract operator, making it scalable for large programs.
Limitations: While highly effective for real-valued variables, the domain struggles with optimal precision for integer variables, as the strong closure does not always find the most precise DBM in the integer lattice. Additionally, the $O(n^3)$ cost, while polynomial, can still be significant for programs with thousands of variables, necessitating further optimization or packing techniques.